# Multi-Tenancy Implementation Guide

## Quick Reference for Future Development

This guide explains the multi-tenancy architecture implemented in Phases 33-34 and how to work with it.

---

## When Adding a New API Route

### ✅ DO THIS:

```
// app/api/items/route.ts
import { safeHandler } from '@/lib/api/safe-handler';
import { ApiErrors } from '@/lib/api/api-response';
import { prisma } from '@/lib/prisma';

export const GET = safeHandler(async (req, context) => {
  // context = { userId, restaurantId, role }

  const items = await prisma.item.findMany({
    where: {
      restaurantId: context.restaurantId, // ✅ Always filter by restaurantId
      active: true,
    },
  });

  return NextResponse.json(items);
});

export const POST = safeHandler(async (req, context) => {
  // Check role before proceeding
  if (context.role === 'COOK') {
    return ApiErrors.FORBIDDEN(); // ✅ Use ApiErrors for consistent responses
  }

  const body = await req.json();

  const item = await prisma.item.create({
    data: {
      ...body,
      restaurantId: context.restaurantId, // ✅ Always set restaurantId
    },
  });

  return NextResponse.json(item, { status: 201 });
});
```

### ❌ DO NOT DO THIS:

```
// ❌ WRONG: Manual session extraction
export async function GET(req) {
  const session = await getServerSession(); // Manual auth
  // Missing restaurantId filtering!
}

// ❌ WRONG: No restaurantId in create
const item = await prisma.item.create({
  data: { name, code } // Where's restaurantId?!
});

// ❌ WRONG: Manual role checking
if (session?.user?.role !== 'MANAGER') { // Inconsistent
  return NextResponse.json({ error: 'Not authorized' });
}
```

---

## Understanding the Context Object

### What is `context`?

Passed automatically to all `safeHandler` routes:

```
interface RestaurantContext {
  userId: string;        // UUID of authenticated user
  restaurantId: string;  // UUID of user's current restaurant
  role: UserRole;        // 'OWNER' | 'ADMIN' | 'MANAGER' | 'CASHIER' | 'COOK'
}
```

### How it’s Extracted

1. Client makes request with NextAuth session cookie
2. Middleware (edge) reads JWT token
3. `safeHandler` wrapper extracts context from session
4. Context passed to your handler

---

## Database Queries with restaurantId

### Find Many (with filtering)

```
const items = await prisma.item.findMany({
  where: {
    restaurantId: context.restaurantId,
    active: true,
  },
});
```

### Find Unique (using compound key)

```
// For models with @@unique([restaurantId, code])
const item = await prisma.item.findUnique({
  where: {
    restaurantId_code: {
      restaurantId: context.restaurantId,
      code: 'ITEM-001',
    },
  },
});
```

### Create (always set restaurantId)

```
const item = await prisma.item.create({
  data: {
    restaurantId: context.restaurantId, // ✅ CRITICAL
    name: 'Tomato',
    code: 'TOM-001',
  },
});
```

### Update (filter by compound key)

```
const updated = await prisma.item.update({
  where: {
    restaurantId_code: {
      restaurantId: context.restaurantId,
      code: 'ITEM-001',
    },
  },
  data: { price: 10.5 },
});
```

### Delete (filter by compound key)

```
const deleted = await prisma.item.delete({
  where: {
    restaurantId_code: {
      restaurantId: context.restaurantId,
      code: 'ITEM-001',
    },
  },
});
```

---

# Error Handling

## Use ApiErrors for Consistency

```
import { ApiErrors } from '@/lib/api/api-response';

// Unauthorized (no session)
return ApiErrors.UNAUTHORIZED();

// Forbidden (wrong role)
return ApiErrors.FORBIDDEN();

// Not found (data doesn't exist)
return ApiErrors.NOT_FOUND();

// Invalid request (bad input)
return ApiErrors.INVALID_REQUEST({ message: 'Email is required' });

// Server error
return ApiErrors.INTERNAL_ERROR();
```

## Response Format

```
// All errors follow this format
{
  "error": "UNAUTHORIZED",
  "message": "Session not found",
  "status": 401
}
```

---

# Role-Based Access Control

## Checking Roles in Handlers

```
export const DELETE = safeHandler(async (req, context) => {
  // Only OWNER can delete
  if (context.role !== 'OWNER') {
    return ApiErrors.FORBIDDEN();
  }

  // Proceed with deletion
  await prisma.item.delete({...});
});

export const POST = safeHandler(async (req, context) => {
  // Multiple roles allowed
  if (!['OWNER', 'MANAGER'].includes(context.role)) {
    return ApiErrors.FORBIDDEN();
  }

  // Proceed with creation
  await prisma.item.create({...});
});
```

### Available Roles

| Role | Permissions |
|---|---|
| **OWNER** | Full access to all resources |
| **ADMIN** | Administrative functions |
| **MANAGER** | Manage operations and staff |
| **CASHIER** | Process transactions |
| **COOK** | Kitchen-only operations (restricted) |

---

## Middleware & Headers

### What the Middleware Does

1. **Extracts JWT** from NextAuth session cookie
2. **Sets authentication headers** for logging
3. **Detects region** based on geolocation
4. **Logs all requests** for audit trail
5. **Sets security headers** (nosniff, XSS protection, etc.)

### Headers Available in Your Handler

```
// From middleware (available in request headers)
request.headers.get('X-User-Id');      // UUID of user
request.headers.get('X-User-Role');    // User's role
request.headers.get('X-Authenticated'); // 'true' or 'false'
request.headers.get('X-Region');       // Detected region (BR, US, etc)
request.headers.get('X-Audit-Log');    // Request log entry
```

---

## Common Patterns

### Fetch with Filtering

```
export const GET = safeHandler(async (req, context) => {
  const { search, status } = req.nextUrl.searchParams;

  const items = await prisma.item.findMany({
    where: {
      restaurantId: context.restaurantId,
      ...(search && { name: { contains: search, mode: 'insensitive' } }),
      ...(status && { status }),
    },
    orderBy: { createdAt: 'desc' },
  });

  return NextResponse.json(items);
});
```

### Create with Related Data

```
export const POST = safeHandler(async (req, context) => {
  const { name, categoryId } = await req.json();

  // Verify category belongs to same restaurant
  const category = await prisma.category.findUnique({
    where: {
      restaurantId_id: {
        restaurantId: context.restaurantId,
        id: categoryId,
      },
    },
  });

  if (!category) {
    return ApiErrors.NOT_FOUND();
  }

  const item = await prisma.item.create({
    data: {
      restaurantId: context.restaurantId,
      name,
      categoryId,
    },
  });

  return NextResponse.json(item, { status: 201 });
});
```

### Update with Audit Log

```
export const PUT = safeHandler(async (req, context) => {
  const { id } = req.params;
  const { price } = await req.json();

  const oldItem = await prisma.item.findUnique({
    where: {
      restaurantId_id: {
        restaurantId: context.restaurantId,
        id,
      },
    },
  });

  if (!oldItem) {
    return ApiErrors.NOT_FOUND();
  }

  const updated = await prisma.item.update({
    where: {
      restaurantId_id: {
        restaurantId: context.restaurantId,
        id,
      },
    },
    data: { price },
  });

  // Log audit trail
  await prisma.auditLog.create({
    data: {
      userId: context.userId,
      action: 'UPDATE',
      entityType: 'Item',
      entityId: id,
      changes: JSON.stringify({ price: { old: oldItem.price, new: price } }),
    },
  });

  return NextResponse.json(updated);
});
```

---

## Prisma Schema Notes

### Models with restaurantId

These models require `restaurantId` in all operations:

- Ingredient
- Recipe
- Supplier
- Stock
- StockMovement
- WasteLog
- ProductionPlan

- ProductionPlanItem
- ShoppingList
- ShoppingListItem
- StaffMember
- IngredientCategory
- And 30+ more...

### Compound Unique Constraints

Example: `@@unique([restaurantId, code])` on Ingredient

Means:
- Code must be unique **within a restaurant**
- Different restaurants can have same code
- Always query with both fields together

---

## Troubleshooting

### “Session not found” Error

**Cause**: User not authenticated or session expired
**Fix**: Ensure user is logged in before making API calls

```
// In client components
const { data: session } = useSession();
if (!session) return <p>Please log in</p>;
```

### “restaurantId does not exist” Error

**Cause**: Forgot to add restaurantId when creating data
**Fix**: Always include `restaurantId: context.restaurantId`

```
// ❌ WRONG
data: { name, code }

// ✅ RIGHT
data: { restaurantId: context.restaurantId, name, code }
```

### “Record not found” on Update

**Cause**: Using wrong unique constraint in where clause
**Fix**: Use compound key with restaurantId

```
// ❌ WRONG
where: { id: itemId }

// ✅ RIGHT
where: {
  restaurantId_id: {
    restaurantId: context.restaurantId,
    id: itemId,
  },
}
```

---

# Testing Multi-Tenancy

## Manual Testing Checklist

- [ ] Create item in Restaurant A
- [ ] Login as user from Restaurant B
- [ ] Verify item is NOT visible in Restaurant B
- [ ] Try to access item directly - should get 404
- [ ] Try to update item from Restaurant B - should get 403 or 404
- [ ] Verify COOK role cannot POST to create items
- [ ] Check X-Audit-Log header in response

## E2E Test Example

```
// tests/multi-tenancy.test.ts
describe('Multi-Tenancy', () => {
  test('User A cannot see User B data', async () => {
    // Create as User A
    const itemA = await fetch('/api/items', {
      method: 'POST',
      headers: { 'Authorization': `Bearer ${tokenA}` },
      body: JSON.stringify({ name: 'Secret Item' }),
    });

    // Try to access as User B
    const response = await fetch(`/api/items/${itemA.id}`, {
      headers: { 'Authorization': `Bearer ${tokenB}` },
    });

    // Should get 404
    expect(response.status).toBe(404);
  });
});
```

---

# Resources

- **Helper Libraries**: `/lib/api/`
- **Middleware**: `/middleware.ts`
- **Implementation Guidelines**: `/lib/api/README.md`

- **Phase 34 Docs**: `/PHASE_34_MIDDLEWARE.md`
- **Phase Summary**: `/PHASE_33-34_SUMMARY.md`

---

## TL;DR

1. Use `safeHandler` for all new routes
2. Always filter queries by `restaurantId: context.restaurantId`
3. Always set `restaurantId: context.restaurantId` when creating data
4. Use `ApiErrors` for consistent error responses
5. Check `context.role` for authorization
6. Use compound keys for unique lookups
7. Let middleware handle authentication and headers

**That's it! Welcome to multi-tenant development!** 🎉